

SESSION 1

Introduction to Django & Setup

Topics Covered: What is Django?, Why Django vs Flask, Django features, MVT Architecture, Installing Django, Creating first Django project, Running Django server

Activities: Create project: doctorfinder_project, Explore project structure

Task 1: Install Django using pip in your local Python environment and verify the installation by running 'django-admin --version' in the terminal.

Explanation & Approach: *Django can be installed via Python's package manager pip. Verifying the version confirms that Django's executable binaries are properly mapped to your system's PATH.*

```
# Step 1: Open Terminal / Command Prompt and install Django
pip install django

# Step 2: Verify installation version
django-admin --version

# Sample Output:
# 5.0.2
```

Task 2: Create a new Django project named 'foodiehub_project' using the django-admin startproject command, then run the development server and open http://127.0.0.1:8000/ to confirm it's working.

Explanation & Approach: *'django-admin startproject' creates the boilerplate directory layout. Running 'manage.py runserver' starts Django's lightweight local WSGI development server.*

```
# Step 1: Create the project
django-admin startproject foodiehub_project

# Step 2: Navigate into the project folder
cd foodiehub_project

# Step 3: Run the local development server
python manage.py runserver

# Output in terminal:
# Watching for file changes with StatReloader
# Performing system checks...
# System check identified no issues (0 silenced).
# Starting development server at http://127.0.0.1:8000/
# Quit the server with CTRL-BREAK.
```

Task 3: Explore the folder structure of your 'foodiehub_project' and write down the purpose of each key file: manage.py, settings.py, urls.py, wsgi.py, and asgi.py.

Explanation & Approach: *Understanding Django's file structure is essential for configuring database settings, routing URLs, and deploying to web servers.*

```
# foodiehub_project Folder Structure Explanation

1. manage.py:
  - Command-line utility to interact with the project (run server, create apps, run migrations, create superuser).

2. settings.py:
  - Main configuration file containing INSTALLED_APPS, DATABASES, TEMPLATES, MIDDLEWARE, STATIC_URL, and SECURITY settings.

3. urls.py:
  - Root URL dispatcher containing URL patterns that map incoming HTTP request paths to views.

4. wsgi.py:
  - Web Server Gateway Interface configuration used for synchronous deployment (e.g., Gunicorn, Apache, uWSGI).

5. asgi.py:
  - Asynchronous Server Gateway Interface configuration used for async Django features (WebSockets, Channels, Async views).
```

Task 4: Write a short comparison (3-4 lines) between Django and Flask, focusing on features and use cases for each. Use an example app you use daily (like Instagram or Zomato) to explain which framework would be better and why.

Explanation & Approach: *Django is a full-stack 'batteries-included' framework with built-in ORM, Admin, and Auth, whereas Flask is a micro-framework providing high flexibility and minimalism.*

```
Comparison: Django vs Flask

- Django is a batteries-included full-stack framework with built-in ORM, Admin panel, Authentication, and Security, while Flask is a lightweight micro-framework offering total flexibility where developers select their own database and form libraries.

- Daily App Example (Zomato / Instagram):
  Django is far better for building complex apps like Zomato or Instagram because it natively provides user role authentication, database ORM relationships (Restaurants, Reviews, Orders), an instant Admin dashboard for managing menu items, and scalable security out of the box.
```